

Reviewer Pack — Minimal Symplectic Leaves and Frege Proof Depth Inequality

Entry bb9f05c55843 · INCONCLUSIVE

SEC autonomous research engine — attribution: Ludovico Kubler

2026-06-04 20:31:23 UTC

Minimal Symplectic Leaves and Frege Proof Depth Inequality

Entry ID: bb9f05c55843 **Verdict:** INCONCLUSIVE **Recorded:** 2026-06-04 20:31:23 UTC

1. Conjecture

Field A (mathematical branch): Symplectic Geometry **Field B** (complexity object): Resolution Proofs (Frege Proof Complexity)

Statement:

For every d -regular graph G , the number of symplectic leaves of its associated moment polytope φ_G is linearly correlated with its Frege proof depth $d(\varphi_G)$, such that the number of symplectic leaves $L(\varphi_G) = \Theta(d(\varphi_G))$.

Rationale (proposer's reasoning):

Symplectic geometry has a rich structure in terms of geometric invariants, and it is believed that these invariants can provide insights into the complexity of computational problems. The moment polytope is a key object in symplectic geometry, and its leaves are known to be related to the topology of the underlying variety. Frege proof depth is a measure of the complexity of proving unsatisfiability for a given formula. By linking these two fields, we may uncover new structural properties that can be exploited in complexity analysis.

Taxonomy category: SymplecticGeometry × ResolutionProofComplexity (status at proposal time:)

2. Pre-registration (Popper-style)

Hash (SHA-256 prefix): f68f292be42ccd6f

This hash commits to the conjecture statement, acceptance criterion, and seed list **before** the empirical test runs. Tampering with the test or its data after this hash was computed would be detectable.

Acceptance criterion:

The conjecture is supported if the Spearman's rank correlation coefficient between the number of symplectic leaves $L(\varphi_G)$ and Frege proof depth $d(\varphi_G)$ for a set of random d -regular graphs G , each with n variables, is at least 0.8, and the mean absolute difference between $L(\varphi_G)$ and $\Theta(d(\varphi_G))$ across all seeds is less than or equal to 3.

3. Barrier filter (F1)

Final: PASS

Barrier	Verdict	Confidence	LLM ₁	LLM ₂
RELATIVIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
NATURAL_PROOFS	SAFE	0.50	UNCERTAIN	UNCERTAIN
ALGEBRIZATION	SAFE	0.50	UNCERTAIN	UNCERTAIN
KARP_LIPTON	SAFE	1.00	SAFE	UNCERTAIN

4. Novelty audit

Verdict: ADJACENT_OK against 1 hits across arXiv + Semantic Scholar + ECCC

Search queries (3): - symplectic geometry AND resolution proofs AND Frege proof depth - moment polytope AND symplectic leaves AND complexity theory - Frege proof depth inequality AND associated graphs AND symplectic geometry

Top relevant hits considered: - [s2:f00d943cea727962981a17afc4650e9510d2008f] How a Math Class Can Be Two Places at Once: A Proposed Mathematical/Philosophical Collaboration

5. Empirical test harness

Multi-seed protocol: 5 seeds (11, 23, 37, 53, 71)

Sandbox: isolated subprocess, stdlib-only, ≤90s wall time per cycle **Execution:** rc=1, elapsed=0.1s

5.1 Generated Python source

```
import random
import math
from fractions import Fraction

def run_trial(seed: int) -> dict:
    random.seed(seed)

    def generate_d_regular_graph(n, d):
        if n % d != 0:
            raise ValueError("Graph size must be a multiple of the degree")

        graph = {i: [] for i in range(n)}
        edges_added = set()

        while len(edges_added) < (n * d) // 2:
            u, v = random.sample(range(n), 2)
            if u != v and (u, v) not in edges_added and (v, u) not in edges_added:
                graph[u].append(v)
                graph[v].append(u)
                edges_added.add((u, v))
                edges_added.add((v, u))

        return graph

    def tseitin_formula(n):
        variables = [f'x{i}' for i in range(1, n+1)]
        clauses = []
```

```

# Clause for each variable being true or false
for var in variables:
    clauses.append([var])
    clauses.append([-var])

# Clause for each pair of variables being different
for i in range(n):
    for j in range(i + 1, n):
        clauses.append([f'x{i}', -f'x{j}'])
        clauses.append([-f'x{i}', f'x{j}'])

return clauses

def frege_proof_depth(clauses):
    # Simplified Frege proof depth calculation
    return len(clauses)

def moment_polytope(graph):
    n = len(graph)
    vertices = [list(range(n))]
    faces = []

    while vertices:
        vertex = vertices.pop()
        if not vertex:
            continue

        face = [vertex[0]]
        for neighbor in graph[vertex[0]]:
            if neighbor not in face:
                face.append(neighbor)

        faces.append(face)
        vertices.extend([face[i+1:] for i in range(len(face)-1)])

    return len(faces)

def spearman_correlation(l1, l2):
    n = len(l1)
    rank_l1 = {x: i for i, x in enumerate(sorted(set(l1)), start=1)}
    rank_l2 = {x: i for i, x in enumerate(sorted(set(l2)), start=1)}

    sum_diff_squares = sum((rank_l1[l1[i]] - rank_l2[l2[i]]) ** 2 for i in range(n))
    sum_rank_diffs = sum(abs(rank_l1[l1[i]] - rank_l2[l2[i]]) for i in range(n))

    return 1 - (6 * sum_diff_squares) / (n * (n**2 - 1))

def mean_absolute_difference(l, theta):
    return sum(abs(x - theta) for x in l) / len(l)

n_values = [5, 10, 15, 20, 30, 40]
results = []

for n in n_values:
    d = random.randint(2, min(n-1, 4))

```

```

graph = generate_d_regular_graph(n, d)
clauses = tseitin_formula(n)

l_phi_g = moment_polytope(graph)
d_phi_g = frege_proof_depth(clauses)

results.append((l_phi_g, d_phi_g))

if not results:
    return {
        "metric_name": "Spearman's rank correlation coefficient",
        "metric_value": None,
        "instances_tested": 0,
        "n_max": 0,
        "conjecture_holds": False,
        "counterexample": "mapping_undefined"
    }

l_phi_g_values, d_phi_g_values = zip(*results)
rho = spearman_correlation(l_phi_g_values, d_phi_g_values)
theta = Fraction(n * (n - 1), 2) / d
mean_abs_diff = mean_absolute_difference(l_phi_g_values, theta)

return {
    "metric_name": "Spearman's rank correlation coefficient",
    "metric_value": rho,
    "instances_tested": len(results),
    "n_max": max(n_values),
    "conjecture_holds": rho >= 0.8 and mean_abs_diff <= 3,
    "counterexample": ""
}

if __name__ == "__main__":
    import sys
    seeds = [int(s) for s in sys.argv[1:]] or [2, 3, 5, 7, 11, 13, 17, 19, 23, 29] * 3

    results = []

    for seed in seeds:
        trial_result = run_trial(seed)
        print(f"TRIAL: {trial_result}")
        results.append(trial_result)

    if all("conjecture_holds" in r and r["conjecture_holds"] for r in results):
        mean_rho = sum(r["metric_value"] for r in results) / len(results)
        std_rho = math.sqrt(sum((r["metric_value"] - mean_rho) ** 2 for r in results) / len(results))
        support_fraction = len([r for r in results if r["conjecture_holds"]]) / len(results)
        print(f"RESULT: SUPPORTED mean={mean_rho} std={std_rho} support_fraction={support_fraction}")
    elif any("counterexample" in r and r["counterexample"] for r in results):
        first_failing_seed = next(r["seed"] for r in results if "counterexample" in r and r["counterexample"])
        print(f"RESULT: FALSIFIED counterexample={r['counterexample']} first_failing_seed={first_failing_seed}")
    else:
        print("RESULT: INCONCLUSIVE no seeds supported the conjecture")

```

6. Per-seed results

(no seed-level data — test crashed before producing TRIAL: lines)

7. Test stdout (last 2KB)

--STDERR--

Traceback (most recent call last):

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f6b95df9.py", line 136, in <module>
    trial_result = run_trial(seed)
                   ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f6b95df9.py", line 97, in run_trial
    graph = generate_d_regular_graph(n, d)
           ~~~~~
```

```
File "/home/ludo/Scrivania/SEC/research/pvsnp_sandbox/test_f6b95df9.py", line 23, in generate_d_re
    raise ValueError("Graph size must be a multiple of the degree")
```

ValueError: Graph size must be a multiple of the degree

8. Critic adversarial review

Critic verdict: CHALLENGE

Critic reasoning:

skipped: test crashed before producing data

9. Final verdict & safety rail

Verdict: INCONCLUSIVE

Reasoning:

The test code crashed before producing data, which prevents us from evaluating the Spearman's rank correlation coefficient and mean absolute difference as required by the support condition. | next: Review the code to identify the cause of the crash and fix it to allow for proper testing.

11. Audit log (LLM calls)

Total LLM calls: 10

#	Phase	Provider	Model	Tokens in	out	Latency (ms)
1	propose	ollama_remote	glm4:latest	0	0	13988
2	propose	ollama_remote	glm4:latest	0	0	13240
3	preregistration	ollama_remote	glm4:latest	0	0	10212
4	novelty	ollama_remote	glm4:latest	0	0	8337
5	novelty	ollama_remote	glm4:latest	0	0	13998
6	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	16562
7	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14605
8	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	14346
9	test_gen	ollama_remote	qwen2.5-coder:7b	0	0	15068
10	judge	ollama_remote	glm4:latest	0	0	11748

Totals: 0 input tokens, 0 output tokens, ~\$0.0000 cost, 132104 ms total latency. Provider mix: {'ollama_remote': 10}

(full prompt+response transcripts available in `research/audit/bb9f05c55843.jsonl`)

12. Reproducibility

To reproduce this cycle:

1. Apply the test harness in section 5.1 with seeds 11,23,37,53,71
2. The Python sandbox is pure-stdlib; any Python 3.11+ should suffice
3. The full LLM transcripts are in `research/audit/bb9f05c55843.jsonl` for verification of provenance
4. The pre-registration hash in section 2 commits to the test design before execution; tampering would be detectable.

Sandbox file: `research/pvsnp_sandbox/test_*.py` (per-cycle)

Replay tarball: `research/replay/bb9f05c55843.tar.gz` (if generated)